

Unitree SDK2 Python 签名包：从零开始指南

Unitree SDK2 Python Bindings

目录

先看结论	5
这个包是什么？	5
这个包不是什么？	6
最容易记住的比喻	6
当前最重要的两个标签	6
五分钟快速开始	6
1. 打开绑定目录	6
2. 创建独立 Python 环境	6
3. 安装签名 wheel	7
4. 安装一个类型检查器	7
5. 新建第一个只做类型检查的程序	7
6. 故意写错一次，观察检查器	8
理解两个安装包	8
签名 wheel	8
运行时扩展	8
为什么签名包和运行时要分开？	9
理解四道可用性关卡	9
第 1 关：签名可见	9
第 2 关：绑定已实现	9
第 3 关：运行时可导入	9
第 4 关：硬件环境可用	9
四关对照表	10
安装开发环境	10
路径 A：只写代码和检查签名	10
路径 B：构建真正的 Linux 运行时	10
配置编辑器和类型检查器	12
VS Code	12
PyCharm	13
Mypy	13
Pylint	13
验证编辑器是否读到了签名	13
模块和命名空间	13
总体结构	13
三类主要 API	14
推荐的导入方式	14
为什么有多个 MotorCmd？	14

IDL 消息入门	14
IDL 是什么?	14
创建消息	15
写入标量字段	15
读取字段	15
写入列表字段	15
比较消息	15
类型正确不等于协议正确	15
列表、定长数组和复制语义	16
什么是复制语义?	16
错误示例: 只改返回的副本	16
正确示例: 读取、修改、写回	16
嵌套对象同样要写回	16
定长数组	17
G1 CRC 工具	17
list 和 Sequence 的区别	18
建议写一个更新辅助函数	18
DDS 入门	18
DDS 是什么?	18
六个基础名词	19
为什么 topic 和消息类型必须同时匹配?	19
初始化 DDS	19
查找 Linux 网卡名称	19
释放 DDS	19
完整生命周期顺序	20
查看运行时支持的消息类型	20
安全订阅机器人状态	20
完整示例	20
每一部分在做什么?	21
queue_length 怎么选?	22
查看最近数据时间	22
如果一直没有输出	22
在自定义主题上发布消息	22
发布示例	22
构造函数的类型约束	23
write() 返回值	23
wait_microsec 是什么?	23
Publisher 属性	23
Robot Client 入门	24
Robot Client 和 Subscriber 有什么区别?	24
当前绑定原则	24
G1 只读查询完整示例	24
每一行的意义	24
G1 当前绑定范围	25
H1 多输出值示例	26
H2 列表输出示例	26

常用通用方法	27
处理状态码和返回元组	27
为什么第一个返回值总是状态码?	27
0 和非零值	27
一个通用的单输出辅助函数	27
多输出不要强行套用二元组函数	28
不要忽略状态码	28
理解回调、线程和 GIL	28
回调在哪里执行?	28
GIL 是什么?	28
为什么只读 Robot 查询会释放 GIL?	28
回调里应该做什么?	29
推荐的“回调只入队”模式	29
回调异常会怎样?	29
关闭顺序为什么重要?	30
运动 API 的安全边界	30
为什么编辑器能看到 <code>move()</code> , 但仍然不能直接试跑?	30
能通过类型检查和属性检查, 也不代表可以安全运行	30
当前分类统计	30
即使名字像“停止”, 也属于运动安全边界	30
上层模块现在应该怎样写?	31
查询完整 API 清单	31
为什么需要 <code>api_manifest.json</code> ?	31
仓库中的位置	32
使用 <code>jq</code> 查询一个方法	32
查询所有当前可用的只读 API	32
查询所有运动 API	32
不安装 <code>jq</code> , 用 Python 查询	33
从已安装的 <code>wheel</code> 定位清单	33
用清单做启动前保护	34
如何阅读 <code>.pyi</code> 文件	34
常见错误与解决办法	34
1. <code>ModuleNotFoundError: No module named 'unitree_sdk2_cpp'</code>	34
2. 编辑器提示找不到导入, 但 Mypy 能通过	35
3. 编辑器有补全, 但运行时报 <code>AttributeError</code>	35
4. CMake 报“不支持当前平台”	35
5. CMake 报找不到 Unitree 库	36
6. 导入时报 <code>libddsc.so</code> 或 <code>libddscxx.so</code> 找不到	36
7. 链接时报静态库不是 PIC	36
8. <code>ValueError</code> 、 <code>TypeError</code> 或 <code>OverflowError</code>	36
9. Subscriber 创建成功, 但收不到数据	37
10. Robot Client 查询超时	37
11. 回调异常只出现在终端, 主程序没有捕获	37
12. 修改 <code>command.motor_cmd[0]</code> 后值没变	37
13. 安装 <code>wheel</code> 时提示文件不存在	37
14. 签名和实际运行时看起来不一致	38

常见问题	38
我只想要所有函数签名，需要远程服务器吗？	38
安装 stub 后能在 macOS 上创建 MotorCmd() 吗？	38
为什么 pip show 成功, import unitree_sdk2_cpp 仍失败？	38
stub wheel 会覆盖真实扩展吗？	38
AVAILABLE 是否表示不连接机器人也能调用？	38
为什么有些构造函数也是 SIGNATURE_ONLY？	38
为什么还要保留 API 安全分类？	38
read-only 是否表示绝对没有任何影响？	39
可以在回调里直接操作 UI 吗？	39
可以在一个进程里创建多个 Subscriber 吗？	39
可以在一个 Subscriber 上更换消息类型吗？	39
ChannelPublisher 为什么要传类而不是对象？	39
64 个消息类都能用于 Publisher 和 Subscriber 吗？	39
能不能运行官方 C++ 示例来验证？	39
为什么文档不提供运动命令示例？	39
如何确认 wheel 没有损坏？	39
API 速查	40
顶层 API	40
Channel API	40
IDL 模块覆盖	41
idl.go2 类索引	41
idl.hg 类索引	41
idl.g1 类索引	42
idl.hg_doubleimu 类索引	42
idl.ros2 类索引	42
Robot 命名空间覆盖	42
当前可构造的 26 个 Client	43
常见只读返回类型	43
Robot 预览类索引	44
版本、覆盖率与验证状态	46
文档对应版本	46
签名覆盖统计	46
状态数字应该怎样理解？	46
Robot JSON 值类型怎样使用？	47
已完成的静态验证	47
Linux 二进制验证边界	47
发布或交付前检查清单	48
推荐学习路线	48
阶段 1：只理解 Python 签名	48
阶段 2：理解消息对象	49
阶段 3：理解 DDS，只使用自定义主题	49
阶段 4：只读设备数据	49
阶段 5：建立工程边界	49
阶段 6：任何运动能力之前	49
最后检查：我现在到底能做什么？	50

只安装了 <code>stub wheel</code>	50
安装了匹配的 Linux 扩展	50
一句话决策规则	50

这份文档面向第一次接触 Python 类型提示、C++ 绑定、DDS 或 Unitree SDK2 的开发者。你不需要先理解 `pybind11`、`CMake` 或 `Clang AST`，也可以使用这个包开始写代码。

本文对应以下本地构建产物：

```
unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

完成本文后，你将能：

- 在 macOS、Windows 或 Linux 上安装签名包；
- 在 VS Code、PyCharm、Mypy 或 Pyright 中获得自动补全和类型检查；
- 分清“编辑器看得见”“当前绑定已实现”“本机能导入”“机器人能响应”四件不同的事；
- 创建和读取 IDL 消息对象；
- 理解 DDS 发布者、订阅者、主题和回调；
- 在 Linux 上构建真正的 `unitree_sdk2_cpp` 扩展；
- 理解如何调用只读 Robot Client，以及为什么运动接口必须有额外安全门；
- 使用 `api_manifest.json` 判断一个 API 是否真的可执行。

[!TIP] 需要逐项查找全部函数、重载、属性、参数和返回值时，请打开 [完整 API 参考](#)。入门指南负责解释概念和学习顺序，API Reference 则像字典一样按模块和类列出全部 1213 个 manifest 条目。

[!WARNING] 机器人是物理设备。错误的控制命令可能导致机器人突然运动、跌倒、碰撞、损坏设备或伤人。当前 binding 已注册运动 API，但“能调用”绝不等于“已对你的机器人和现场验证为安全”。本文不提供可直接执行的运动控制示例，也不要自定义发布示例改成机器人控制主题。默认测试不会构造 Client、初始化 DDS 或发送任何机器人指令。

先看结论

这个包是什么？

`unitree_sdk2_cpp_stubs` 是 `unitree_sdk2_cpp` 的 Python 类型签名包，也叫 `stub` 包或 `.pyi` 包。

它的主要作用是告诉编辑器和类型检查器：

- 有哪些模块；
- 有哪些类；
- 一个方法接收什么参数；
- 一个方法返回什么类型；
- 哪些成员是属性；
- 哪些 API 当前已绑定，哪些只是未来接口预览。

安装以后，你可以在代码中输入：

```
from unitree_sdk2_cpp.idl.go2 import MotorCmd

motor = MotorCmd()
motor.q = 0.0
motor.kp = 20.0
```

编辑器会知道 `q` 和 `kp` 是 `float`，也会在你拼错名字或传错类型时提示。

这个包不是什么？

它不是机器人模拟器，不包含真正的 C++ 运行时代码，也不会让 macOS 或 Windows 突然具备运行 Unitree Linux SDK 的能力。

只安装签名包以后：

```
python -c "import unitree_sdk2_cpp"
```

仍然可能出现：

```
ModuleNotFoundError: No module named 'unitree_sdk2_cpp'
```

这是预期行为，不代表签名包安装失败。签名包服务于编辑器和静态检查器；真正执行程序需要另外安装 Linux 二进制扩展。

最容易记住的比喻

可以把这两个包想成“说明书”和“机器”：

组件	类比	能做什么	不能做什么
unitree_sdk2_cpp_stubs	说明书	补全、跳转、显示参数、静态检查	不能执行 C++，不能连接机器人
unitree_sdk2_cpp	真正的机器	创建消息、运行 DDS、调用已绑定客户端	不能自动提供尚未实现的 API

当前最重要的两个标签

每个预览接口都会被标为以下状态之一：

标签	含义	能否假设当前二进制里存在？
AVAILABLE	当前绑定源代码已经实现	可以，但仍需通过平台、导入和硬件条件
SIGNATURE_ONLY	只有类型签名，用于提前开发	不可以

[!IMPORTANT] SIGNATURE_ONLY 不是“可能偶尔能用”，而是“不要按当前运行时可用来写执行逻辑”。它可以通过 Mypy/Pyright，不代表 Python 运行时存在这个属性。

五分钟快速开始

这一节只安装签名包，不连接机器人，不发送网络数据，适合所有操作系统。

1. 打开绑定目录

从仓库根目录执行：

```
cd unitree_sdk2_bindings
```

确认 wheel 文件存在：

```
ls dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

你应该看到同名文件路径。

2. 创建独立 Python 环境

推荐 Python 3.10 或更高版本。

使用标准 venv：

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell 对应命令是：

```
py -3.10 -m venv .venv
.venv\Scripts\Activate.ps1
```

如果你使用 Conda：

```
conda create -n unitree-py310 python=3.10 -y
conda activate unitree-py310
```

[!TIP] 命令行前面出现 `(.venv)` 或 `(unitree-py310)`，通常表示环境已经激活。后续始终使用 `python -m pip`，可以减少“装到了另一个 Python”这类问题。

3. 安装签名 wheel

```
python -m pip install \
    dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

检查安装结果：

```
python -m pip show unitree-sdk2-cpp-stubs
```

预期能看到：

```
Name: unitree-sdk2-cpp-stubs
Version: 0.3.0
```

4. 安装一个类型检查器

以下二选一即可。本文优先用 Mypy 演示：

```
python -m pip install mypy
```

或者：

```
python -m pip install pyright
```

5. 新建第一个只做类型检查的程序

创建 `signature_demo.py`：

```
from unitree_sdk2_cpp.idl.go2 import LowCmd, MotorCmd

motor = MotorCmd()
motor.mode = 0
motor.q = 0.0
motor.dq = 0.0
motor.tau = 0.0
motor.kp = 0.0
motor.kd = 0.0
motor.reserve = [0, 0, 0]

command = LowCmd()
motors = command.motor_cmd
motors[0] = motor
command.motor_cmd = motors

print(type(command).__name__)
```

只做静态检查：

```
python -m mypy --strict signature_demo.py
```

预期输出：

```
Success: no issues found in 1 source file
```

[!CAUTION] 如果你只安装了签名 wheel，不要执行 `python signature_demo.py`。静态检查会读取 `.pyi`，实际执行则需要 Linux 二进制扩展。

6. 故意写错一次，观察检查器

把：

```
motor.q = 0.0
```

暂时改成：

```
motor.q = "zero"
```

再次执行 Mypy，应该得到类似：

```
error: Incompatible types in assignment
```

这就是签名包最直接的价值：许多错误在代码运行前就能被发现。

理解两个安装包

签名 wheel

文件：

```
dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

文件名各部分的含义：

部分	含义
unitree_sdk2_cpp_stubs	Python 分发包名称
0.3.0	签名包版本
py3	适用于 Python 3 的类型信息
none-any	不依赖操作系统或 CPU 架构

因此，这个 wheel 可以安装到 macOS、Windows、Linux `x86_64` 或 Linux `aarch64` 的开发环境中。

运行时扩展

真实运行时导入名称是：

```
import unitree_sdk2_cpp
```

它由 C++/pybind11 编译而成，文件名通常类似：

```
unitree_sdk2_cpp.cpython-310-aarch64-linux-gnu.so
```

这个文件不是纯 Python 文件。文件名会包含 Python ABI、CPU 架构和 Linux 平台信息，不能随意从一个平台复制到另一个平台。

当前仓库中的 Unitree SDK 和 CycloneDDS 库是 Linux ELF 文件，因此运行时构建支持：

- Linux `x86_64`；
- Linux `aarch64`。

当前不支持直接在以下平台构建运行时：

- macOS；
- Windows；
- 非 `x86_64/aarch64` 的 Linux 架构。

为什么签名包和运行时要分开？

常见 workflow 是：

```
macOS / Windows 开发机
└─ 安装签名 wheel
   └─ 写 Python 代码
   └─ 获得自动补全
   └─ 运行 Mypy / Pyright

Linux x86_64 / aarch64 目标机
└─ 安装真实扩展 + 签名 wheel
   └─ 执行 Python 程序
   └─ 使用 DDS
   └─ 在明确授权和安全条件下访问机器人
```

这样，即使目标机器人或 Linux 服务器暂时不可用，上层模块也可以先按完整签名开发。

理解四道可用性关卡

判断某段代码能不能真正运行时，请按顺序检查四道关卡。

```
第 1 关：签名可见
↓
第 2 关：绑定已实现
↓
第 3 关：运行时可导入
↓
第 4 关：网络、DDS、服务和硬件可用
```

第 1 关：签名可见

问题：编辑器或类型检查器是否知道这个 API？

由签名 wheel 决定。只要 .pyi 里有声明，编辑器就可能显示它。

通过这一关只说明“可以看见”，不说明“可以执行”。

第 2 关：绑定已实现

问题：当前 pybind11 源码是否注册了这个 API？

查看该方法的标签：

- AVAILABLE：当前绑定源码已经实现；
- SIGNATURE_ONLY：当前只有声明，没有运行时实现。

第 3 关：运行时可导入

问题：当前机器是否安装了匹配的 Linux 扩展和动态库？

可以用以下命令检查：

```
python -c "import unitree_sdk2_cpp; print(unitree_sdk2_cpp.__file__)"
```

只有这一命令成功，才说明 Python 找到了真正的二进制模块。

第 4 关：硬件环境可用

问题包括：

- DDS 是否使用正确的 domain ID；
- 是否选择了连接机器人的网络接口；
- 主题名和消息类型是否匹配；
- 机器人相关服务是否启动；

- 机器人型号和固件 API 是否匹配;
- 防火墙、交换机或虚拟机网络是否允许 DDS 流量;
- 当前操作是否经过现场安全确认。

四关对照表

场景	签名可见	绑定已实现	运行时可导入	硬件可用	结果
macOS 只装 stub	是	未验证	否	否	可补全和静态检查
Linux 装扩展, 调用 SIGNATURE_ONLY	是	否	是	可能是	运行时仍会失败
Linux 装扩展, 无机器人, 创建 MotorCmd	是	是	是	不需要	可以运行
Linux 装扩展, 订阅状态但网卡错了	是	是	是	否	通常收不到数据
Linux 装扩展, 正确连接后只读查询	是	是	是	是	可以获得结果

[!NOTE] “Mypy 通过” 只覆盖第一关。它不会连接机器人, 也不会检查 .so 文件中是否真的有这个方法。

安装开发环境

路径 A: 只写代码和检查签名

适合:

- macOS;
- Windows;
- 暂时没有 Linux 服务器;
- 暂时不连接机器人;
- 先开发上层业务模块。

最低要求:

- Python 3.10 或更高版本;
- 签名 wheel;
- Mypy、Pyright 或支持 Python 类型提示的 IDE。

安装命令:

```
cd unitree_sdk2_bindings
python3 -m venv .venv
source .venv/bin/activate
python -m pip install \
    dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
python -m pip install mypy
```

这条路径不需要 CMake、C++ 编译器、Clang、DDS 网卡或机器人。

路径 B: 构建真正的 Linux 运行时

适合:

- Ubuntu/Linux x86_64;
- Ubuntu/Linux aarch64;
- 需要实际创建消息、初始化 DDS 或运行只读 Robot Client。

1. 检查系统架构

```
uname -s
uname -m
```

支持的典型输出是：

```
Linux
x86_64
```

或者：

```
Linux
aarch64
```

2. 准备 Python 环境

Conda 示例：

```
conda create -n unitree-bindings python=3.10 -y
conda activate unitree-bindings
```

标准 venv 示例：

```
python3 -m venv .venv-linux
source .venv-linux/bin/activate
```

3. 安装构建依赖

如果系统允许使用 APT：

```
sudo apt-get update
sudo apt-get install -y build-essential cmake ninja-build python3-dev
```

安装 Python 构建依赖：

```
python -m pip install --upgrade pip
python -m pip install \
    "scikit-build-core>=0.10" \
    "pybind11>=2.12" \
    wheel
```

4. 确认 SDK 静态库和 DDS 库存在

在仓库根目录检查当前架构对应的文件。例如 aarch64：

```
ls lib/aarch64/libunitree_sdk2.a
ls thirdparty/lib/aarch64/libddsc.so
ls thirdparty/lib/aarch64/libddscxx.so
```

x86_64 则把目录名换成 x86_64。

5. 安装真实扩展

```
cd unitree_sdk2_bindings
python -m pip install -e . --no-build-isolation
```

这里各参数的含义：

参数	含义
-e	可编辑安装，源码改动后便于重新构建
.	使用当前目录中的 pyproject.toml
--no-build-isolation	使用当前环境已经安装的构建依赖

普通用户不需要安装 Clang，也不需要重新扫描 C++ 头文件，因为生成后的绑定源码已经提交在仓库中。

只有维护绑定生成器时才需要：

```
python -m pip install -e . \
    --config-settings=cmake.define.UNITREE_REGENERATE_BINDINGS=ON
```

该模式会额外依赖 clang++，初学者不要从这里开始。

6. 安装签名 wheel

真实扩展负责执行，签名 wheel 负责补全。两者可以安装在同一个环境中：

```
python -m pip install \
    dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

7. 先做不接触机器人的导入检查

```
python -c \
    "import unitree_sdk2_cpp as u; print(u.OsHelper.instance().get_hostname())"
```

这一步只读取本机主机名，不初始化 DDS，不控制机器人。

再检查 64 种已注册消息类型：

```
python -c \
    "from unitree_sdk2_cpp import channel;
    print(len(channel.registered_message_types()))"
```

当前预期输出：

```
64
```

8. 运行默认安全测试

```
python -m pip install pytest
python -m pytest -q
```

默认测试不会初始化 DDS、联系硬件或发送运动命令。带有 `dds`、`hardware`、`motion` 标记的测试需要显式条件，不应在普通开发环境中自动运行。

[!IMPORTANT] 本文对应的完整 Robot Client 包装器源码已经生成，但当前修订仍应在目标 Linux 架构上完成一次新的编译、导入和符号检查后，再把它视为二进制验证完成。

配置编辑器和类型检查器

VS Code

1. 安装 Microsoft Python 扩展和 Pylance。
2. 打开仓库目录。
3. 在命令面板中选择 Python: Select Interpreter。
4. 选择刚才安装签名 wheel 的 `.venv` 或 Conda 环境。
5. 重新加载窗口。

可以在项目根目录创建 `pyrightconfig.json`：

```
{
    "typeCheckingMode": "strict",
    "venvPath": ".",
    "venv": ".venv",
    "pythonVersion": "3.10"
}
```

如果使用 Conda，通常由 VS Code 直接选择 Conda 解释器即可，不必硬编码 `venvPath`。

PyCharm

1. 打开 **Settings / Preferences**。
2. 进入 **Project > Python Interpreter**。
3. 选择安装过 `unitree-sdk2-cpp-stubs` 的解释器。
4. 等待索引完成。
5. 在导入、类名或方法名上使用跳转定义，即可看到 `.pyi` 签名。

Mypy

最简单的命令：

```
python -m mypy --strict your_program.py
```

项目级配置可以写入 `pyproject.toml`：

```
[tool.mypy]
python_version = "3.10"
strict = true
warn_unused_ignores = true
show_error_codes = true
```

然后运行：

```
python -m mypy src tests
```

Pyright

```
pyright your_program.py
```

或者检查整个项目：

```
pyright
```

验证编辑器是否读到了签名

新建：

```
from unitree_sdk2_cpp.robot.g1 import LocoClient

client = LocoClient()
reveal_type(client.get_fsm_id())
```

Mypy 应显示：

```
Revealed type is "tuple[builtins.int, builtins.int]"
```

删除 `reveal_type` 后再提交业务代码；它是类型检查器的调试指令，不是正常运行逻辑。

模块和命名空间

总体结构

```
unitree_sdk2_cpp
├── OsHelper
├── channel
│   ├── initialize
│   ├── initialize_from_config
│   ├── release
│   ├── registered_message_types
│   ├── ChannelPublisher[T]
│   └── ChannelSubscriber[T]
├── idl
└── go2
```

```

|   |   | g1
|   |   | hg
|   |   | hg_doubleimu
|   |   | ros2
|   |   └─ robot
|   |       | 通用 Client / ClientBase 类型
|   |       | a2
|   |       | as2
|   |       | b2
|   |       | g1
|   |       | go2
|   |       | h1
|   |       | h2
|   |       └─ r1

```

三类主要 API

分类	用途	是否通常需要 DDS/机器人
idl.*	创建和读取消息数据	创建对象不需要；收发需要
channel	初始化 DDS，发布或订阅消息	需要网络环境
robot.*	调用机器人服务客户端	通常需要 DDS、网络 and 对应服务

推荐的导入方式

明确导入你需要的类型：

```

from unitree_sdk2_cpp import channel
from unitree_sdk2_cpp.idl.go2 import LowState, MotorCmd
from unitree_sdk2_cpp.robot.g1 import LocoClient

```

不推荐：

```

from unitree_sdk2_cpp.idl.go2 import *

```

明确导入更容易阅读，也能避免不同模块中同名的 `MotorCmd`、`LowState` 等类型互相覆盖。

为什么有多个 `MotorCmd`？

不同机器人消息命名空间可能有同名类型：

```

from unitree_sdk2_cpp.idl.go2 import MotorCmd as Go2MotorCmd
from unitree_sdk2_cpp.idl.hg import MotorCmd as HgMotorCmd

```

它们来自不同 C++/IDL 定义，字段和数组长度不一定相同，不能因为 Python 类名相同就混用。

IDL 消息入门

IDL 是什么？

IDL 是 Interface Definition Language，中文常译为“接口定义语言”。在这里，它描述 DDS 消息的结构。你可以把一个 IDL 消息理解成一张固定格式的表格：

```

MotorCmd
├─ mode: int
├─ q: float
├─ dq: float
├─ tau: float
├─ kp: float
├─ kd: float
└─ reserve: list[int]

```

发送方和接收方必须对这张“表格”的格式达成一致，DDS 才能正确编码和解码数据。

创建消息

```
from unitree_sdk2_cpp.idl.go2 import MotorCmd

motor = MotorCmd()
```

`MotorCmd()` 调用无参数构造函数，创建一个由 C++ 消息对象支持的 Python 对象。

写入标量字段

```
motor.mode = 0
motor.q = 1.25
motor.dq = 0.0
motor.tau = 0.0
motor.kp = 10.0
motor.kd = 0.5
```

字段含义取决于具体机器人协议。类型签名只告诉你 Python 类型，不替代机器人型号对应的协议说明、单位定义和安全限制。

读取字段

```
position = motor.q
gain = motor.kp

print(position)
print(gain)
```

类型检查器知道：

```
position: float
gain: float
```

写入列表字段

```
motor.reserve = [0, 0, 0]
```

setter 的类型是 `Sequence[int]`，因此列表和元组都可以作为输入：

```
motor.reserve = (0, 0, 0)
```

读取时统一得到新的 Python `list[int]`：

```
values = motor.reserve
```

比较消息

IDL 消息绑定实现了 `==` 和 `!=`：

```
from unitree_sdk2_cpp.idl.go2 import MotorCmd

left = MotorCmd()
right = MotorCmd()

print(left == right)
right.q = 1.0
print(left != right)
```

比较的是消息内容，不是 Python 变量名。

类型正确不等于协议正确

下面的代码对类型检查器来说可能完全正确：

```
motor.mode = 999999
```

但 C++ 底层字段可能是 `uint8_t`、`int16_t` 或其他范围有限的整数。运行时可能抛出 `OverflowError`，即使

没有抛出，数值也未必符合设备协议。

[!IMPORTANT] Python 签名中的 `int` 只表示“整数”，不表达 C++ 整数的位宽、正负范围或枚举语义。写入真实消息前，必须查对应 SDK 头文件和机器人协议文档。

列表、定长数组和复制语义

这一节非常重要。很多第一次使用 C++ 绑定的人，会在这里遇到“我明明改了列表，为什么原消息没变”的问题。

什么是复制语义？

当前消息字段采用复制语义：

- 从消息读取数组、vector 或嵌套消息时，Python 得到一份副本；
- 修改这份副本，不会立即修改原 C++ 消息；
- 要让修改生效，必须把修改后的值重新赋回字段。

这样做的原因是避免 Python 持有可能已经失效的 C++ 内存引用。

错误示例：只改返回的副本

```
from unitree_sdk2_cpp.idl.go2 import LowCmd

command = LowCmd()
command.motor_cmd[0].q = 1.0
```

这行代码先读取 `command.motor_cmd` 的副本，再修改副本中的元素。临时副本随后被丢弃，原来的 `command` 不一定发生任何变化。

正确示例：读取、修改、写回

```
from unitree_sdk2_cpp.idl.go2 import LowCmd

command = LowCmd()

motors = command.motor_cmd
motors[0].q = 1.0
command.motor_cmd = motors
```

把它记成三个动作：

1. 取出来
2. 改副本
3. 赋回去

嵌套对象同样要写回

`LowCmd.bms_cmd` 是一个嵌套的 `BmsCmd`：

```
from unitree_sdk2_cpp.idl.go2 import LowCmd

command = LowCmd()

bms = command.bms_cmd
bms.off = 0
command.bms_cmd = bms
```

不要假设下面的链式修改会保留：

```
command.bms_cmd.off = 0
```


定长数组

Python 签名把 C++ `std::array<T, N>` 表示为读取时的 `list[T]` 和写入时的 `Sequence[T]`。但底层长度仍然是固定的。

例如 `Go2 LowCmd.motor_cmd` 对应：

```
std::array<MotorCmd, 20>
```

因此必须写回正好 20 个 `MotorCmd`。

安全的更新方式是从现有字段读取完整列表，只替换需要的元素：

```
motors = command.motor_cmd
assert len(motors) == 20

motors[0].q = 1.0
command.motor_cmd = motors
```

如果传入错误长度：

```
command.motor_cmd = []
```

运行时通常会抛出长度不匹配异常。静态类型检查器只知道这是 `Sequence[MotorCmd]`，无法从普通 `list` 类型判断长度是否等于 20。

G1 的 `LowCmd.motor_cmd` 使用相同规则，但长度是 35：

```
from unitree_sdk2_cpp.idl.g1 import LowCmd, MotorCmd

command = LowCmd()
command.motor_cmd = [MotorCmd() for _ in range(35)]
```

`idl.g1` 中的 13 个消息类是 `idl.hg` 对应 C++ DDS 类型的易读别名。例如，`idl.g1.LowCmd` 与 `idl.hg.LowCmd` 是同一个 Python/C++ 类型，不会产生一份不兼容的重复消息定义。

G1 CRC 工具

G1 低层命令和状态使用 SDK 自带的 CRC 算法。绑定直接调用 C++ `crc32_core`，没有在 Python 中重新实现算法：

```
from unitree_sdk2_cpp.idl.g1 import (
    LowCmd,
    MotorCmd,
    compute_crc,
    update_crc,
    validate_crc,
)

command = LowCmd()
command.motor_cmd = [MotorCmd() for _ in range(35)]

crc = compute_crc(command)
written_crc = update_crc(command)

assert written_crc == crc
assert command.crc == crc
assert validate_crc(command)
```

函数区别：

函数	接受类型	是否修改对象	返回值
<code>compute_crc(message)</code>	<code>LowCmd</code> 或 <code>LowState</code>	否	计算出的 <code>int</code> CRC
<code>update_crc(message)</code>	<code>LowCmd</code>	是，写入 <code>message.crc</code>	写入的 <code>int</code> CRC
<code>validate_crc(message)</code>	<code>LowCmd</code> 或 <code>LowState</code>	否	当前 CRC 是否匹配

[!WARNING] 计算或写入 CRC 只是在内存中准备消息，不会自动发布。CRC 正确也不表示命令在机械、电气或现场条件下安全；不要把这个示例接到 G1 低层控制 topic。

list 和 Sequence 的区别

签名常见形式：

```
@property
def reserve(self) -> list[int]: ...

@reserve.setter
def reserve(self, value: Sequence[int]) -> None: ...
```

含义是：

- 读取结果保证是可修改的 Python 列表；
- 写入参数只要求是一个按顺序提供元素的容器；
- 因此写入时可传列表或元组；
- 字符串虽然也是序列，但不应当传给整数序列字段。

建议写一个更新辅助函数

下面的函数只修改 Python 消息对象，不会发送 DDS 数据：

```
from unitree_sdk2_cpp.idl.go2 import LowCmd

def set_motor_position(
    command: LowCmd,
    motor_index: int,
    position: float,
) -> None:
    motors = command.motor_cmd

    if not 0 <= motor_index < len(motors):
        raise IndexError(f"motor_index out of range: {motor_index}")

    motors[motor_index].q = position
    command.motor_cmd = motors
```

它展示了两个好习惯：

- 修改定长数组前检查索引；
- 遵守“读取、修改、写回”的复制语义。

[!WARNING] 这个函数只用于解释消息编辑方式。不要把构造出的低层控制消息发布到机器人控制主题，除非你已经拥有对应型号的完整协议、安全方案、急停措施和现场授权。

DDS 入门

DDS 是什么？

DDS 是 Data Distribution Service。你可以先把它理解成一个局域网里的实时消息总线：



DDS 不要求发布者直接知道订阅者的 IP 和端口。双方主要通过主题、消息类型、domain 和网络环境互相

发现。

六个基础名词

名词	初学者理解	在本绑定中的对应项
Domain	一组彼此可发现的 DDS 程序	<code>domain_id</code>
Network interface	DDS 使用哪张网卡	<code>network_interface</code>
Topic	消息频道名称	<code>topic: str</code>
Message type	频道中消息的结构	<code>LowState</code> 、 <code>String</code> 等类
Publisher	向主题发送消息	<code>ChannelPublisher</code>
Subscriber	从主题接收消息	<code>ChannelSubscriber</code>

为什么 topic 和消息类型必须同时匹配？

topic 名可以看作“频道名”，消息类型可以看作“频道格式”。

```
频道名相同 + 类型相同    -> 可以通信
频道名不同               -> 不会互相收到
频道名相同 + 类型不同    -> 不兼容，不能按预期通信
```

仅仅知道 `rt/lowstate` 这个字符串还不够，还要知道它使用哪个 IDL 类型。

初始化 DDS

最常见的初始化形式：

```
from unitree_sdk2_cpp import channel

channel.initialize(domain_id=0, network_interface="eth0")
```

参数说明：

参数	类型	默认值	含义
<code>domain_id</code>	<code>int</code>	<code>0</code>	DDS domain 编号
<code>network_interface</code>	<code>str</code>	<code>""</code>	网卡名称；空字符串交给底层配置处理

也可以使用配置文件：

```
channel.initialize_from_config(config_file="cyclonedds.xml")
```

两种初始化方式通常二选一，不要在同一进程中反复混用。

查找 Linux 网卡名称

```
ip -brief link
```

可能看到：

```
lo            UNKNOWN
eth0          UP
wlan0         UP
```

连接机器人的有线网卡可能叫 `eth0`、`enp3s0`、`eno1` 等。名称不是固定的，不能照抄别人的机器配置。

释放 DDS

程序结束前调用：

```
channel.release()
```

推荐使用 `try/finally`，保证发生异常时也释放：

```

from unitree_sdk2_cpp import channel

channel.initialize(0, "eth0")

try:
    # 创建 publisher、subscriber 或 client
    ...
finally:
    channel.release()

```

完整生命周期顺序

```

1. channel.initialize(...)
2. 创建 Publisher / Subscriber / Robot Client
3. 对具体对象调用 init_channel() / init()
4. 收发消息或进行只读查询
5. close_channel()
6. channel.release()

```

[!IMPORTANT] 先关闭 publisher/subscriber, 再调用全局 `channel.release()`。不要在回调仍可能执行时销毁共享状态。

查看运行时支持的消息类型

真实 Linux 扩展安装后可以执行：

```

from unitree_sdk2_cpp import channel

for name in channel.registered_message_types():
    print(name)

```

当前显式注册了 64 种消息类型。运行时不能仅凭任意 Python 类去实例化一个新的 C++ DDS 模板类型；消息类型必须存在于这份注册表中。

安全订阅机器人状态

本节示例只读取 Go2 低层状态，不发布控制数据。

[!WARNING] 这段代码需要已经构建的 Linux 扩展、正确的 DDS 网络 and 对应机器人。它不适用于只安装签名 wheel 的 macOS/Windows 环境。

完整示例

```

from __future__ import annotations

import threading

from unitree_sdk2_cpp import channel
from unitree_sdk2_cpp.idl.go2 import LowState

def on_low_state(message: LowState) -> None:
    """DDS 每收到一条状态消息，就调用一次这个函数。"""
    try:
        motors = message.motor_state

        if not motors:
            print("收到状态消息，但 motor_state 为空")
            return

        first_motor = motors[0]
        print(
            "motor[0]",
            f"q={first_motor.q:.4f}",
            f"dq={first_motor.dq:.4f}",

```

```

        f"temperature={first_motor.temperature}",
    )
except Exception as exc:
    # 不要让异常逃出 DDS 工作线程。
    print(f"处理 LowState 失败: {exc!r}")

def main() -> None:
    network_interface = "eth0" # 改成实际连接机器人的网卡名

    channel.initialize(domain_id=0, network_interface=network_interface)

    subscriber = channel.ChannelSubscriber(
        "rt/lowstate",
        LowState,
        on_low_state,
        queue_length=10,
    )

    try:
        subscriber.init_channel()
        print("正在等待 LowState: 按 Ctrl+C 退出")

        stop_event = threading.Event()
        stop_event.wait()
    except KeyboardInterrupt:
        print("正在退出")
    finally:
        subscriber.close_channel()
        channel.release()

if __name__ == "__main__":
    main()

```

每一部分在做什么？

导入消息类型：

```
from unitree_sdk2_cpp.idl.go2 import LowState
```

这同时告诉运行时和类型检查器：订阅者预期收到 `LowState`。

定义回调：

```
def on_low_state(message: LowState) -> None:
```

回调接收一个参数，没有返回值。每次 DDS 收到新样本，C++ 层会复制出一个由 Python 安全持有的消息对象，然后调用这个函数。

创建订阅者：

```
subscriber = channel.ChannelSubscriber(
    "rt/lowstate",
    LowState,
    on_low_state,
    queue_length=10,
)
```

四个参数分别是：

1. topic 名；
2. 消息类型类本身，不是 `LowState()` 对象；
3. 回调函数；
4. 队列长度。

启动订阅：

```
subscriber.init_channel()
```

创建 Python 对象本身不等于 DDS 通道已经启动，必须显式初始化。

保持进程存活：

```
stop_event.wait()
```

如果主程序立即结束，订阅者也会随进程退出，自然收不到后续数据。

queue_length 怎么选？

当前签名是：

```
ChannelSubscriber(
    topic,
    message_type,
    callback,
    queue_length=0,
)
```

初学阶段可以使用默认值 0，或根据数据频率和处理速度使用一个小的正数，例如 10。队列并不是越大越好：

- 队列太小，处理跟不上时可能丢掉旧样本；
- 队列太大，可能积压过时状态并增加内存占用；
- 对高频状态，通常比起处理每个历史样本，更重要的是及时看到最新状态。

具体行为还取决于底层 DDS QoS，不能只靠这一参数推断全部策略。

查看最近数据时间

```
timestamp = subscriber.last_data_available_time
print(timestamp)
```

这是底层记录的最近一次数据可用时间。其精确单位和时间基准应以 SDK/CycloneDDS 实现为准，不要直接把它当作 Unix 秒时间使用。

如果一直没有输出

按这个顺序检查：

1. `subscriber.init_channel()` 是否执行；
2. 进程是否仍在运行；
3. 网卡名是否正确；
4. 机器人是否连接在这张网卡上；
5. `domain_id` 是否匹配；
6. `topic` 是否为该型号和固件实际使用的名字；
7. `LowState` 是否为这个 `topic` 的真实消息类型；
8. 防火墙或虚拟机网络是否拦截 DDS；
9. 回调内部是否抛出异常；
10. 机器人相关服务是否正在发布该主题。

在自定义主题上发布消息

发布者本身是通用能力，但发布到机器人控制主题会产生物理风险。以下示例只在自定义教程主题 `tutorial/hello` 上发布 ROS2 `String` 消息。

[!CAUTION] 不要把示例 `topic` 替换成 `rt/lowcmd`、运动服务主题或任何设备控制主题。一个 `topic` 名看似只是字符串，但它可能连接到真实执行器。

发布示例

```

from unitree_sdk2_cpp import channel
from unitree_sdk2_cpp.idl.ros2 import String

def main() -> None:
    channel.initialize(domain_id=0, network_interface="eth0")

    publisher = channel.ChannelPublisher(
        "tutorial/hello",
        String,
    )

    try:
        publisher.init_channel()

        message = String()
        message.data = "hello from unitree_sdk2_cpp"

        accepted = publisher.write(message, wait_microsec=0)
        print(f"write accepted: {accepted}")
    finally:
        publisher.close_channel()
        channel.release()

if __name__ == "__main__":
    main()

```

构造函数的类型约束

```
publisher = channel.ChannelPublisher("tutorial/hello", String)
```

这里的 `String` 是消息类。由于 `ChannelPublisher` 是泛型，类型检查器会推断：

```
ChannelPublisher[String]
```

因此下面是正确的：

```
publisher.write(String())
```

下面则应当被类型检查器拒绝：

```

from unitree_sdk2_cpp.idl.go2 import LowState

publisher.write(LowState())

```

write() 返回值

签名：

```
def write(self, message: MessageT, wait_microsec: int = 0) -> bool: ...
```

返回的 `bool` 表示底层写操作是否被接受。它不等于“所有订阅者已经处理完成”，也不等于“机器人已经执行”。DDS 发布通常是异步分发过程。

wait_microsec 是什么？

这是微秒级等待参数。默认值 `0` 表示使用默认的非额外等待行为。是否需要非零值取决于底层通道实现和具体场景，初学阶段不要随意设置很大的值。

Publisher 属性

```

print(publisher.topic)
print(publisher.message_type_name)

```

它们可以帮助日志记录和排错，但不代替对端兼容性检查。

Robot Client 入门

Robot Client 和 Subscriber 有什么区别？

Subscriber 更像持续收听广播：

状态发布者 -> topic -> 你的回调

Robot Client 更像一次请求和一次响应：

你的程序 -> 请求 -> 机器人服务
你的程序 <- 响应 <- 机器人服务

很多 `get_*` 方法会等待服务返回，因此可能阻塞一小段时间，也可能超时。

当前绑定原则

当前运行时覆盖 25 个具体 SDK Client 和 `LeaseClient`，包括直接方法、输出参数包装和两个配置回调。

- 340 个公开 Client 方法中有 336 个运行时入口；
- 50 个只读方法全部可用，C++ 输出引用会转换成 Python 返回值；
- 220 个运动方法和 36 个其他硬件副作用方法已经注册，但默认测试只做反射检查，绝不执行；
- 尚未绑定的是抽象 `ClientBase::Init` 和 3 个内部 `ClientStub` 请求方法。

G1 只读查询完整示例

[!WARNING] 这段代码不发运动命令，但需要真实 Linux 扩展、正确网络、G1 和对应服务。请先在不影响现场作业的条件下测试。

```
from unitree_sdk2_cpp import channel
from unitree_sdk2_cpp.robot.g1 import LocoClient

def main() -> None:
    channel.initialize(domain_id=0, network_interface="eth0")

    client = LocoClient()
    client.set_timeout(5.0)

    try:
        client.init()

        status, fsm_id = client.get_fsm_id()

        if status != 0:
            raise RuntimeError(
                f"get_fsm_id failed with status {status}"
            )

        print(f"current fsm_id: {fsm_id}")
    finally:
        channel.release()

if __name__ == "__main__":
    main()
```

每一行的意义

初始化 DDS：

```
channel.initialize(0, "eth0")
```

Robot Client 通过 SDK 的 DDS/RPC 通道工作，因此通常要先初始化全局 channel。

创建具体型号的客户端：

```
client = LocoClient()
```

`g1.LocoClient` 和 `h1.LocoClient` 是不同类型。应从目标型号的命名空间导入。

G1 当前绑定范围

`robot.g1` 暴露 4 个具体 SDK Client:

Client	公开方法数 主要用途
AgvClient	3 G1-D AGV 移动和高度调整
AudioClient	7 TTS、音量、音频流和 LED
G1ArmActionClient	5 预设/自定义手臂动作和动作列表
LocoClient	34 FSM、站立高度、速度、步态和只读状态

这里的“公开方法数”包含 `init()`。运动或硬件副作用方法虽然已经绑定，仍分别标记为 `MOTION_COMMAND` 或 `HARDWARE_SIDE_EFFECT`，默认测试不会调用。

G1 还提供 7 个官方运行安全检查：

```
from unitree_sdk2_cpp.robot.g1 import (
    ang_vel_out_of_limit,
    bad_orientation,
    joint_vel_out_of_limit,
    lost_connection,
    low_battery,
    motor_casing_overheat,
    motor_winding_overheat,
)
```

函数	默认阈值 True 的含义
<code>bad_orientation(low_state, limit_angle=1.0)</code>	1.0 rad 机体姿态倾角过大
<code>joint_vel_out_of_limit(low_state, limit_vel=10.0)</code>	10.0 至少一个关节速度超限
<code>ang_vel_out_of_limit(low_state, limit_vel=6.0)</code>	6.0 IMU 至少一个角速度分量超限
<code>motor_winding_overheat(low_state, limit_temp=120.0)</code>	120.0 至少一个电机绕组过热
<code>motor_casing_overheat(low_state, limit_temp=85.0)</code>	85.0 至少一个电机外壳过热
<code>low_battery(bms_state, limit_soc=20.0)</code>	20.0% 电池 SOC 低于阈值
<code>lost_connection(subscriber, timeout_ms=1000)</code>	1000 ms LowState 订阅数据已经超时

这些函数返回诊断布尔值，不会自己发送阻尼、停机或力矩命令。上层程序必须根据机器人模式和经过验证的故障处理流程决定后续操作。`lost_connection` 只接受消息类型为 `idl.g1.LowState` 的 `ChannelSubscriber`；传入其他消息订阅器会抛出 `TypeError`。

设置超时：

```
client.set_timeout(5.0)
```

单位是秒。还可以使用微秒版本：

```
client.set_timeout_microseconds(5_000_000)
```

不要同时写两个不同值；选择一种表达方式即可。

初始化客户端：

```
client.init()
```

构造对象和初始化服务通道是两步。漏掉 `init()` 通常会导致查询失败。

解包返回值：

```
status, fsm_id = client.get_fsm_id()
```

C++ 原接口是：

```
int GetFsmId(int& fsm_id);
```

在 Python 中不需要先创建一个“输出参数”。绑定把状态码和输出值一起返回：

```
tuple[int, int]
```

H1 多输出值示例

```
from unitree_sdk2_cpp.robot.h1 import LocoClient

client = LocoClient()
client.set_timeout(5.0)
client.init()

status, x, y, yaw = client.get_odom()

if status == 0:
    print(f"x={x:.3f}, y={y:.3f}, yaw={yaw:.3f}")
else:
    print(f"get_odom failed: {status}")
```

C++ 的多个输出引用会按原顺序出现在 Python 元组中，状态码始终排在第一个位置。

H2 列表输出示例

```
from unitree_sdk2_cpp.robot.h2 import LocoClient

client = LocoClient()
client.set_timeout(5.0)
client.init()

status, ids, names = client.get_available_fsm_ids()

if status == 0:
    for fsm_id, name in zip(ids, names, strict=False):
        print(f"{fsm_id}: {name}")
```

返回类型是：

```
tuple[int, list[int], list[str]]
```

`ids` 和 `names` 来自两个 C++ 输出 `vector`。正常情况下它们应当对应，但稳健的业务代码仍应检查长度：

```
if len(ids) != len(names):
    raise RuntimeError("FSM id/name count mismatch")
```

常用通用方法

具体 Client 通常继承以下可用方法：

方法	返回类型	用途
set_timeout(seconds)	None	设置秒级等待超时
set_timeout_microseconds(value)	None	设置微秒级等待超时
get_lease_id()	int	读取 lease ID
get_api_version()	str	读取本地客户端 API 版本
get_server_api_version()	str	查询服务端 API 版本

是否能成功查询服务端版本仍取决于机器人服务和网络。

处理状态码和返回元组

为什么第一个返回值总是状态码？

Unitree C++ 客户端大量使用这种形式：

```
int GetSomething(OutputType& output);
```

返回的 int 表示操作状态，output 引用接收数据。Python 不需要引用参数，所以绑定转换为：

```
status, output = client.get_something()
```

0 和非零值

SDK 示例通常把：

- 0 当作成功；
- 非 0 当作错误或其他状态。

基础写法：

```
status, value = client.get_fsm_id()

if status == 0:
    print(value)
else:
    print(f"query failed: {status}")
```

生产代码应保留原始状态码，结合具体 Client 的 SDK 错误码定义解释，而不是把所有非零值都改写成同一个模糊错误。

一个通用的单输出辅助函数

```
from typing import TypeVar

T = TypeVar("T")

def require_ok(result: tuple[int, T], operation: str) -> T:
    status, value = result

    if status != 0:
        raise RuntimeError(
            f"{operation} failed with status {status}"
        )

    return value
```

使用：

```
fsm_id = require_ok(client.get_fsm_id(), "get_fsm_id")
volume = require_ok(audio_client.get_volume(), "get_volume")
```

这个辅助函数只适用于“状态码 + 一个输出值”的二元组。

多输出不要强行套用二元组函数

对于：

```
status, x, y, yaw = h1_client.get_odom()
```

直接检查更清楚：

```
if status != 0:
    raise RuntimeError(f"get_odom failed with status {status}")

position = (x, y, yaw)
```

不要忽略状态码

不推荐：

```
_, fsm_id = client.get_fsm_id()
print(fsm_id)
```

如果请求失败，输出值可能没有业务意义。类型正确不代表查询成功。

理解回调、线程和 GIL

这一节解释运行行为，不要求你会写 C++。

回调在哪里执行？

DDS 收到数据后，通常在底层工作线程中触发回调，而不是等 Python 主线程主动来取。

```
DDS 工作线程收到样本
↓
C++ 把样本复制到 Python 可持有的对象
↓
C++ 获取 Python GIL
↓
调用你的 Python callback(message)
↓
回调返回，释放 GIL
```

GIL 是什么？

GIL 是 CPython 的 Global Interpreter Lock。简单理解：底层 C++ 线程要执行 Python 代码前，必须先获得进入 Python 解释器的许可。

绑定会在调用回调时正确获取 GIL。你不需要自己操作 GIL。

为什么只读 Robot 查询会释放 GIL？

Robot 查询可能等待网络响应。如果等待期间一直占着 GIL，其他 Python 线程就难以继续执行。

当前只读查询包装器在等待 C++ SDK 时会释放 GIL，返回 Python 前再恢复。这样其他 Python 线程仍有机会工作。

这不意味着 Client 本身自动变成线程安全，也不意味着可以从多个线程无保护地并发调用同一实例。

回调里应该做什么？

适合：

- 读取少量字段；
- 做轻量校验；
- 把数据放入线程安全队列；
- 更新简单统计；
- 捕获并记录异常。

不适合：

- 长时间阻塞；
- 大量磁盘 I/O；
- 在回调里等待另一个可能依赖当前 DDS 线程的操作；
- 不受控地创建线程；
- 执行复杂模型推理；
- 抛出未捕获异常。

推荐的“回调只入队”模式

```
from __future__ import annotations

import queue

from unitree_sdk2_cpp.idl.go2 import LowState

state_queue: queue.Queue[LowState] = queue.Queue(maxsize=1)

def on_low_state(message: LowState) -> None:
    try:
        state_queue.put_nowait(message)
    except queue.Full:
        try:
            state_queue.get_nowait()
        except queue.Empty:
            pass

        try:
            state_queue.put_nowait(message)
        except queue.Full:
            pass
```

这个例子保留最新状态，避免慢处理逻辑堵住 DDS 回调。业务线程可以单独调用：

```
latest_state = state_queue.get(timeout=1.0)
```

根据应用需要，也可以保存更多历史样本，但要明确内存上限和丢弃策略。

回调异常会怎样？

绑定不会让 Python 异常穿过 C++ DDS 工作线程边界。未捕获异常会按“不可抛出异常”方式报告，容易变成日志中的异步错误。

因此推荐在回调最外层捕获：

```
def callback(message: LowState) -> None:
    try:
        process(message)
    except Exception:
        logger.exception("failed to process LowState")
```

关闭顺序为什么重要？

如果先销毁 Python 共享对象或先释放全局 DDS，而回调线程仍可能进入，就会出现竞态条件。

推荐顺序：

1. 通知业务线程停止
2. `close_channel()` 停止 subscriber
3. 等业务线程结束
4. `channel.release()`
5. 退出进程

运动 API 的安全边界

为什么编辑器能看到 `move()`，但仍然不能直接试跑？

签名包的目标之一，是给上层模块提供完整的 SDK 设计期视图。因此它收录了运动方法签名，例如某些 Client 中的：

```
def move(self, vx: float, vy: float, vyaw: float) -> int: ...
```

当前对应文档标记为：

AVAILABLE | MOTION_COMMAND

两个标签分别说明：

- AVAILABLE：当前 binding 中已经注册这个 Python 方法；
- MOTION_COMMAND：它属于可能导致物理运动的命令。

能通过类型检查和属性检查，也不代表可以安全运行

这段代码可能通过 Mypy：

```
from unitree_sdk2_cpp.robot.g1 import LocoClient

client = LocoClient()
result = client.move(0.1, 0.0, 0.0)
```

如果安装了匹配的 Linux 扩展，这个方法会真正进入 C++ SDK，并可能向实体机器人发送请求。这里没有初始化 DDS、确认控制权、限制速度、检查姿态，也没有故障处理，因此只能作为签名说明，不能直接执行。

当前分类统计

Robot API 分类中包括：

分类	方法数 当前策略
只读查询	50 全部进入运行时，其中 45 个使用输出参数包装
初始化/生命周期	32 除抽象基类和内部 ClientStub 外均按实际 Client 暴露
其他硬件副作用	38 36 个进入运行时；ClientStub 的 2 个内部请求方法未绑定
运动命令	220 全部进入运行时，保留 MOTION_COMMAND 安全标签

此外，签名清单还包含值类型、构造函数、内部辅助类型等条目，所以不能把上表简单相加当作全部 manifest 条目数。

即使名字像“停止”，也属于运动安全边界

`stop_move()`、`damp()`、`zero_torque()` 等名字听起来像安全操作，但它们仍会改变机器人执行状态，因此仍标为 MOTION_COMMAND。软件方法不能替代物理急停。

不要把“停止”方法当作软件层面的通用急停。真正的安全系统需要：

- 独立且经过验证的物理急停手段；
- 机器人周围的隔离区域；
- 固定或吊装措施；
- 现场观察人员；
- 明确的控制权和 lease 管理；
- 速度、力矩、姿态和工作空间限制；
- 通信丢失后的故障安全策略；
- 对目标型号和固件的专项验证。

上层模块现在应该怎样写？

如果你正在开发运动模块，仍应先把真实调用隔离起来：

1. 使用签名完成接口定义和静态检查；
2. 把硬件调用封装在清晰边界后；
3. 在无硬件环境中用你自己的 fake/mock 测试业务逻辑；
4. 默认 CI 只使用 fake，禁止访问 DDS 和硬件；
5. 只有完成目标机二进制验证、独立安全评审和现场审批后，才启用真实后端。

一个简单的上层抽象示例：

```
from typing import Protocol

class VelocityController(Protocol):
    def set_velocity(
        self,
        vx: float,
        vy: float,
        omega: float,
        duration: float,
    ) -> int: ...

class RecordingController:
    """开发期 fake：只记录参数，不连接机器人。"""

    def __init__(self) -> None:
        self.calls: list[tuple[float, float, float, float]] = []

    def set_velocity(
        self,
        vx: float,
        vy: float,
        omega: float,
        duration: float,
    ) -> int:
        self.calls.append((vx, vy, omega, duration))
        return 0
```

这能让路径规划、参数校验和状态机逻辑先开发，但不会误调用真实设备。

[!WARNING] 不要为了“让示例先跑起来”而使用 `getattr`、`Any` 或忽略类型错误绕过可用性边界。签名预览和真实硬件能力之间必须保留明确的人工审核点。

查询完整 API 清单

为什么需要 `api_manifest.json`？

`.pyi` 适合编辑器阅读，`api_manifest.json` 适合脚本、审计工具和 CI 阅读。

它记录每个条目的：

字段	含义
python_path	完整 Python 路径
cpp_class	对应的 C++ 类或绑定类型
cpp_signature	原始 C++ 签名摘要
status	AVAILABLE 或 SIGNATURE_ONLY
safety	安全/用途分类
binding_strategy	方法采用的绑定策略；仅部分条目包含
python_return	Python 返回类型；仅部分方法条目包含

示例条目：

```
{
  "binding_strategy": "OUTPUT_WRAPPER",
  "cpp_class": "unitree::robot::g1::LocoClient",
  "cpp_signature": "GetFsmId(int &)",
  "python_path": "unitree_sdk2_cpp.robot.g1.LocoClient.get_fsm_id",
  "python_return": "tuple[int, int]",
  "safety": "READ_ONLY",
  "status": "AVAILABLE"
}
```

仓库中的位置

```
unitree_sdk2_bindings/
├── stubs/
│   └── src/
│       ├── unitree_sdk2_cpp-stubs/
│       └── api_manifest.json
```

使用 jq 查询一个方法

从仓库根目录执行：

```
jq '
  .entries[]
  | select(
    .python_path
    == "unitree_sdk2_cpp.robot.g1.LocoClient.get_fsm_id"
  )
' \
unitree_sdk2_bindings/stubs/src/unitree_sdk2_cpp-stubs/api_manifest.json
```

查询所有当前可用的只读 API

```
jq -r '
  .entries[]
  | select(
    .status == "AVAILABLE"
    and .safety == "READ_ONLY"
  )
  | .python_path
' \
unitree_sdk2_bindings/stubs/src/unitree_sdk2_cpp-stubs/api_manifest.json
```

查询所有运动 API

只列出名字进行审计，不要执行：

```
jq -r '
  .entries[]
  | select(.safety == "MOTION_COMMAND")
  | [.status, .python_path]
  | @tsv
' \
unitree_sdk2_bindings/stubs/src/unitree_sdk2_cpp-stubs/api_manifest.json
```


当前 220 个运动方法都应同时保留 `MOTION_COMMAND` 标签并标记为 `AVAILABLE`。下面的命令只审计元数据，绝不执行这些方法：

```
jq -e '
[
  .entries[]
| select(
  .safety == "MOTION_COMMAND"
  and .status == "AVAILABLE"
)
]
| length == 220
'
```

unitree_sdk2_bindings/stubs/src/unitree_sdk2_cpp-stubs/api_manifest.json

成功时输出：

```
true
```

不安装 jq, 用 Python 查询

```
from __future__ import annotations

import json
from pathlib import Path
from typing import Any

manifest_path = Path(
    "unitree_sdk2_bindings/stubs/src/"
    "unitree_sdk2_cpp-stubs/api_manifest.json"
)

manifest: dict[str, Any] = json.loads(
    manifest_path.read_text(encoding="utf-8")
)

target = "unitree_sdk2_cpp.robot.g1.LocoClient.get_fsm_id"

for entry in manifest["entries"]:
    if entry["python_path"] == target:
        print(json.dumps(entry, ensure_ascii=False, indent=2))
        break
else:
    raise SystemExit(f"API not found: {target}")
```

这个程序读取 JSON，不导入 `unitree_sdk2_cpp`，所以可以在 macOS、Windows 或没有真实扩展的环境中执行。

从已安装的 wheel 定位清单

如果手边没有源码仓库，可以通过分发包元数据找到清单：

```
from __future__ import annotations

import json
from importlib.metadata import distribution
from typing import Any

dist = distribution("unitree-sdk2-cpp-stubs")
files = dist.files or []

manifest_file = next(
    file
    for file in files
    if str(file).endswith("api_manifest.json")
)
```

```
manifest_path = dist.locate_file(manifest_file)

with manifest_path.open(encoding="utf-8") as stream:
    manifest: dict[str, Any] = json.load(stream)

print(manifest["summary"])
```

用清单做启动前保护

对于设计期插件系统，可以先判断目标 API 是否为 AVAILABLE：

```
from __future__ import annotations

from collections.abc import Iterable, Mapping
from typing import Any

def require_available(
    entries: Iterable[Mapping[str, Any]],
    python_path: str,
) -> None:
    for entry in entries:
        if entry.get("python_path") != python_path:
            continue

        if entry.get("status") != "AVAILABLE":
            raise RuntimeError(
                f"{python_path} is {entry.get('status')}, not AVAILABLE"
            )

    return

raise LookupError(f"API is absent from manifest: {python_path}")
```

[!NOTE] 这只能验证“当前绑定源码宣称可用”。它仍不能替代导入测试、服务探测和硬件安全确认，也不能防止签名 wheel 与运行时扩展版本不匹配。

如何阅读 .pyi 文件

.pyi 只描述接口，不包含实现。例如：

```
class LocoClient(Client):
    def get_fsm_id(self) -> tuple[int, int]:
        """AVAILABLE | READ_ONLY | OUTPUT_WRAPPER."""
        ...
```

逐项解释：

片段	含义
<code>class LocoClient(Client)</code>	<code>LocoClient</code> 继承通用 <code>Client</code>
<code>self</code>	当前对象，调用时不用手动传
<code>-> tuple[int, int]</code>	返回两个整数
第一个 <code>int</code>	SDK 状态码
第二个 <code>int</code>	FSM ID 输出值
<code>AVAILABLE</code>	当前绑定源码已经实现
<code>READ_ONLY</code>	分类为只读查询
<code>OUTPUT_WRAPPER</code>	C++ 输出引用被转换成 Python 元组
<code>...</code>	stub 没有函数体，真实实现位于二进制扩展

常见错误与解决办法

1. ModuleNotFoundError: No module named 'unitree_sdk2_cpp'

只安装了签名 wheel

这是最常见原因。签名包不会创建可执行模块。

检查：

```
python -m pip show unitree-sdk2-cpp-stubs
python -m pip show unitree-sdk2-cpp
```

如果只是 macOS/Windows 开发环境：

- 不要运行依赖扩展的程序；
- 使用 Mypy/Pyright 检查；
- 在 Linux 目标机执行集成测试。

如果是受支持的 Linux：按[安装开发环境](#)中的路径 B 构建真实扩展。

安装到了另一个 Python

检查解释器和 pip 是否对应：

```
which python
python -m pip --version
python -m pip list
```

始终用：

```
python -m pip install ...
```

不要混用裸 pip、系统 Python、Conda Python 和不同虚拟环境。

2. 编辑器提示找不到导入，但 Mypy 能通过

通常是编辑器选择了错误的解释器。

解决步骤：

1. 在终端运行 `python -m pip show unitree-sdk2-cpp-stubs`；
2. 记录该环境的 Python 路径；
3. 在编辑器中选择同一个解释器；
4. 重载编辑器窗口；
5. 等待类型索引完成。

3. 编辑器有补全，但运行时报 `AttributeError`

例如：

```
AttributeError: 'LocoClient' object has no attribute 'move'
```

通常说明该方法是 `SIGNATURE_ONLY`。在编辑器悬停文档或 `api_manifest.json` 中确认状态。

不要用 `# type: ignore` 解决。类型检查没有错，错的是把设计期签名当成当前运行时能力。

4. CMake 报“不支持当前平台”

典型信息：

```
The bundled Unitree SDK2 and CycloneDDS libraries are Linux ELF binaries
```

原因：正在 macOS、Windows 或不支持的 CPU 上构建。

解决：

- 本机只安装 stub 做开发；
- 把运行时构建放到 Linux x86_64 或 aarch64；
- 不要尝试把 Linux .so 直接加载到 macOS/Windows。

5. CMake 报找不到 Unitree 库

典型信息：

```
Required Unitree library not found
```

检查仓库是否完整，以及 CPU 架构目录是否存在：

```
uname -m
ls lib
ls thirdparty/lib
```

当前构建需要：

```
lib/<arch>/libunitree_sdk2.a
thirdparty/lib/<arch>/libddsc.so
thirdparty/lib/<arch>/libddscxx.so
```

6. 导入时报 libddsc.so 或 libddscxx.so 找不到

当前 wheel 安装逻辑会把 CycloneDDS 共享库放到扩展旁边的 `.libs`，并把运行时搜索路径设置为 `$ORIGIN/.libs`。

优先重新构建并重新安装当前版本：

```
python -m pip uninstall -y unitree-sdk2-cpp
python -m pip install -e . --no-build-isolation
```

Linux 上可检查依赖：

```
python -c "import unitree_sdk2_cpp; print(unitree_sdk2_cpp.__file__)"
ldd /path/to/unitree_sdk2_cpp.cpython-310-*.so
```

ldd 中不应出现：

```
not found
```

不要把永久修改全局 `LD_LIBRARY_PATH` 当作首选修复，它容易掩盖 wheel 打包或 `RPATH` 问题。

7. 链接时报静态库不是 PIC

如果看到类似：

```
relocation ... can not be used when making a shared object;
recompile with -fPIC
```

说明 `libunitree_sdk2.a` 中至少有对象文件不是位置无关代码。Python 扩展是共享对象，链接进来的静态库需要满足 PIC 要求。

这不是 Python 代码问题，也不能靠 Mypy 修复。需要由 SDK 库提供方或构建维护者提供使用 `-fPIC` 编译的静态库。

8. ValueError、TypeError 或 OverflowError

常见原因：

- 向 `float` 字段传了字符串；
- 向嵌套消息字段传了错误类；
- 定长数组长度不匹配；
- Python 整数超出 C++ 字段范围；
- Publisher 的消息对象类型和构造时的类型不同。

先用 Mypy/Pyright 发现类型错误，再检查 C++ 位宽和数组长度。

9. Subscriber 创建成功，但收不到数据

按以下表格排查：

检查项	检查方法
网卡	<code>ip -brief address</code> ，确认连接机器人的接口
链路	检查接口是否为 UP，IP 是否在正确网段
domain	与设备和其他 SDK 示例保持一致
topic	对照目标型号/固件，不要只凭猜测
类型	topic 和 IDL 类型必须一致
初始化	确认 <code>channel.initialize()</code> 和 <code>init_channel()</code> 都已调用
进程寿命	主线程不能立即退出
回调	回调最外层捕获并记录异常
防火墙	检查 DDS 发现和数据流量是否被阻断
虚拟化	容器/虚拟机需要合适的网络模式和组播支持

10. Robot Client 查询超时

可能原因：

- 没有先调用 `channel.initialize()`；
- 没有调用 `client.init()`；
- 网卡或 domain 错误；
- 对应机器人服务未运行；
- 导入了错误型号命名空间下的 Client；
- 客户端与服务端 API/固件不兼容；
- 网络丢包或防火墙阻断；
- timeout 太短。

不要只是一味增加 timeout。先确认服务发现和型号匹配，再适度调整：

```
client.set_timeout(5.0)
```

11. 回调异常只出现在终端，主程序没有捕获

DDS 回调运行在底层工作线程中，异常不会正常传播到主线程的 `try/except`。

在回调自身内部捕获：

```
def callback(message: LowState) -> None:
    try:
        process(message)
    except Exception:
        logger.exception("DDS callback failed")
```

12. 修改 `command.motor_cmd[0]` 后值没变

这是复制语义，不是随机故障。使用：

```
motors = command.motor_cmd
motors[0].q = 1.0
command.motor_cmd = motors
```

13. 安装 wheel 时提示文件不存在

先确认当前目录：

```
pwd
ls dist
```

如果你在仓库根目录，wheel 的相对路径是：

```
unitree_sdk2_bindings/dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

如果你已经进入 `unitree_sdk2_bindings`，相对路径才是：

```
dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

14. 签名和实际运行时看起来不一致

确认两个分发版版本和来源：

```
python -m pip show unitree-sdk2-cpp-stubs
python -m pip show unitree-sdk2-cpp
```

当前 stub 0.3.0 描述的是当前仓库生成时的接口快照。未来运行时版本更新后，应同时重新生成并发布匹配的 stub，不要长期混用不同修订。

常见问题

我只想要所有函数签名，需要远程服务器吗？

不需要。签名 wheel 是 `py3-none-any`，可以在本地安装。所有 `.pyi` 和 `api_manifest.json` 都在 wheel 中。

安装 stub 后能在 macOS 上创建 `MotorCmd()` 吗？

不能实际创建。编辑器会理解这行代码，Mypy/Pyright 可以检查它，但执行构造函数需要真实 Linux 扩展。

为什么 `pip show` 成功，`import unitree_sdk2_cpp` 仍失败？

因为 `pip show unitree-sdk2-cpp-stubs` 证明的是“说明书已安装”，不是“运行时已安装”。查看 `pip show unitree-sdk2-cpp` 才是在检查运行时分发版。

stub wheel 会覆盖真实扩展吗？

它遵循 PEP 561 的 `*-stubs` 布局，用于给同名运行时模块提供类型信息。正常情况下两者可以共存：扩展负责运行，stub 负责类型分析。

AVAILABLE 是否表示不连接机器人也能调用？

不一定。

- 创建 `MotorCmd()` 只需要真实扩展；
- `registered_message_types()` 只需要真实扩展；
- `ChannelSubscriber.init_channel()` 需要 DDS 环境；
- `LocoClient.get_fsm_id()` 还需要正确网络、机器人和服务。

AVAILABLE 只回答“绑定源码有没有实现”。

为什么有些构造函数也是 SIGNATURE_ONLY？

完整签名预览包含 SDK 中许多辅助类、服务类和值对象。当前运行时并没有绑定全部类，所以连构造函数也可能只存在于 `.pyi` 中。

为什么还要保留 API 安全分类？

方法进入运行时只说明 `pybind11` 可以调用它，不说明调用是无副作用或安全的。生成器会保留 `READ_ONLY`、`INITIALIZATION`、`HARDWARE_SIDE_EFFECT` 和 `MOTION_COMMAND` 分类；测试、Agent 和上层应用必须根据分类建立执行权限，不能只看方法名。

read-only 是否表示绝对没有任何影响？

它表示业务意图是查询，不命令机器人动作。它仍会：

- 初始化网络对象；
- 发送服务查询；
- 消耗少量带宽和服务资源；
- 受到超时、服务状态和版本兼容性影响。

因此“只读”不等于“完全没有网络活动”。

可以在回调里直接操作 UI 吗？

通常不建议。GUI 框架一般要求只从主线程更新界面。把消息放入线程安全队列，再让 UI 主线程定期读取。

可以在一个进程里创建多个 Subscriber 吗？

API 允许创建多个实例。每个实例应保存在变量中，并在结束时逐个 `close_channel()`。实际数量、频率和 QoS 仍受资源与 DDS 配置限制。

可以在一个 Subscriber 上更换消息类型吗？

构造时消息类型已经确定。需要另一种类型时，关闭旧订阅者并创建新实例。

ChannelPublisher 为什么要传类而不是对象？

底层需要根据消息类选择预先生成的 C++ 模板注册项。随后 `write()` 才接收具体消息对象。

64 个消息类都能用于 Publisher 和 Subscriber 吗？

当前 64 个 IDL 类都进入了显式 typed channel 注册表。但能否与外部程序通信还取决于对方是否使用同一 topic、同一 DDS 类型和兼容 QoS。

能不能运行官方 C++ 示例来验证？

可以在受控 Linux/硬件环境中参考，但不要默认运行低层控制、运动或音频/灯光等有副作用示例。先从导入、消息构造、注册表检查和只读订阅开始。

为什么文档不提供运动命令示例？

因为这些方法已经能进入真实 C++ SDK，并可能影响物理设备。一个安全运动脚本必须同时知道目标型号、固件、控制模式、初始姿态、限值、场地、固定方式、lease 和物理急停；通用文档无法替你确认这些条件。API Reference 只展示准确签名，并明确标出危险等级。

如何确认 wheel 没有损坏？

在仓库根目录执行：

```
shasum -a 256 \  
  unitree_sdk2_bindings/dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

当前预期 SHA-256:

```
126c3e43656de48a69f946a8d9faea87d396c983f91e4bfee48762144fac76ec
```

Linux 上也可以用：

```
sha256sum \  
  unitree_sdk2_bindings/dist/unitree_sdk2_cpp_stubs-0.3.0-py3-none-any.whl
```

API 速查

这一节用于定位模块，不替代 IDE 中的完整 `.pyi` 签名和 `api_manifest.json`。全部函数签名、每个参数、返回值、C++ 来源和用法见 [完整 API 参考](#)。

顶层 API

```
import unitree_sdk2_cpp

helper = unitree_sdk2_cpp.OsHelper.instance()
```

OsHelper 当前声明：

方法	返回类型	说明
<code>instance()</code>	<code>OsHelper</code>	获取单例
<code>get_uid()</code>	<code>int</code>	当前用户 ID
<code>get_gid()</code>	<code>int</code>	当前组 ID
<code>get_user()</code>	<code>str</code>	当前用户名
<code>get_processor_number()</code>	<code>int</code>	处理器数量
<code>get_page_size()</code>	<code>int</code>	内存页大小
<code>get_hostname()</code>	<code>str</code>	主机名

这些方法读取本机操作系统信息，不访问机器人。

Channel API

```
from unitree_sdk2_cpp import channel
```

全局函数：

签名	说明
<code>registered_message_types() -> list[str]</code>	返回注册的 DDS 消息类型名
<code>initialize(domain_id=0, network_interface="") -> None</code>	按 domain 和网卡初始化
<code>initialize_from_config(config_file="") -> None</code>	按配置文件初始化
<code>release() -> None</code>	释放全局 channel 资源

Publisher:

```
ChannelPublisher(topic: str, message_type: type[MessageT])
```

成员	类型/签名
<code>topic</code>	<code>str</code> , 只读属性
<code>message_type_name</code>	<code>str</code> , 只读属性
<code>init_channel()</code>	<code>-> None</code>
<code>write(message, wait_microsec=0)</code>	<code>-> bool</code>
<code>close_channel()</code>	<code>-> None</code>

Subscriber:

```
ChannelSubscriber(
    topic: str,
    message_type: type[MessageT],
    callback: Callable[[MessageT], None],
```



```
    queue_length: int = 0,
)
```

成员	类型/签名
topic	str, 只读属性
message_type_name	str, 只读属性
last_data_available_time	int, 只读属性
init_channel()	-> None
close_channel()	-> None

IDL 模块覆盖

Python 模块	类数	属性数 主要用途
idl.go2	26	181 Go2 消息
idl.g1	13 个别名	对应 idl.hg G1 友好命名和 CRC 工具
idl.hg	13	81 HG 系列消息
idl.hg_doubleimu	1	6 double IMU 消息
idl.ros2	24	73 常用 ROS2 风格消息
底层类型合计	64	341 全部进入 typed channel 注册表; G1 别名不重复计数

idl.go2 类索引

AudioData
BmsCmd
BmsState
ConfigChangeStatus
Error
Go2FrontVideoData
HeightMap
IMUState
InterfaceConfig
LidarState
MotorCmd
LowCmd
MotorState
LowState
MotorCmds
MotorStates
PathPoint
Req
Res
SportModeCmd
TimeSpec
SportModeState
UwbState
UwbSwitch
VoxelMapCompressed
WirelessController

idl.hg 类索引

AgvBmsState
BmsCmd
BmsState
MotorCmd
HandCmd
IMUState
MotorState
PressSensorState
HandState
LowCmd
LowState
MainBoardState

SportModeState

idl.g1 类索引

idl.g1 提供以下 idl.hg 类型的同对象别名:

```
AgvBmsState
BmsCmd
BmsState
HandCmd
HandState
IMUState
LowCmd
LowState
MainBoardState
MotorCmd
MotorState
PressSensorState
SportModeState
```

模块函数为 `compute_crc()`、`update_crc()` 和 `validate_crc()`。其中 `compute_crc()` 与 `validate_crc()` 各有 `LowCmd`、`LowState` 两个重载。

idl.hg_doubleimu 类索引

doubleIMUState

注意该名称首字母确实是小写 d，应按签名原样导入：

```
from unitree_sdk2_cpp.idl.hg_doubleimu import doubleIMUState
```

idl.ros2 类索引

```
Time
Header
Quaternion
Vector3
Imu
Point
Pose
MapMetaData
OccupancyGrid
PoseWithCovariance
Twist
TwistWithCovariance
Odometry
Point32
PointField
PointCloud2
PointStamped
Pose2D
PoseStamped
PoseWithCovarianceStamped
QuaternionStamped
String
TwistStamped
TwistWithCovarianceStamped
```

Robot 命名空间覆盖

Manifest 中的 Robot 类数:

命名空间	类数
robot 根命名空间	18
robot.a2	8
robot.as2	2

命名空间	类数
robot.b2	25
robot.g1	11 类, 另有 InternalFsmMode 枚举
robot.go2	37
robot.h1	4
robot.h2	8
robot.r1	8
合计	121 类

当前可构造的 26 个 Client

下表列出当前绑定源代码中 AVAILABLE 的 25 个具体 SDK Client 和 LeaseClient。最后一列只列专属只读查询；没有只读查询不表示命令方法不可用，完整方法以 API Reference 和 manifest 为准。

命名空间	Client	当前专属只读方法
robot	LeaseClient	get_id()、applied()
a2	AudioClient	get_volume()
a2	SportClient	get_state()
as2	SportClient	get_state()
b2	BackVideoClient	get_image_sample()
b2	ConfigClient	get(name)
b2	FrontVideoClient	get_image_sample()
b2	MotionSwitcherClient	check_mode()、get_silent()
b2	RobotStateClient	service_list()、get_pkg_version()
b2	SportClient	无专属只读方法
g1	AgvClient	无专属只读方法
g1	AudioClient	get_volume()
g1	G1ArmActionClient	get_action_list()
g1	LocoClient	get_fsm_id()、get_fsm_mode()、get_balance_mode()、get_swing_height()、get_stand_height()、get_phase()、get_mimic_motion()
go2	ConfigClient	get(name)
go2	ObstaclesAvoidClient	无专属只读方法
go2	RobotStateClient	service_list()
go2	SportClient	无专属只读方法
go2	UtrackClient	is_tracking()
go2	VideoClient	get_image_sample()
go2	VuiClient	get_switch()、get_volume()、get_brightness()
h1	LocoClient	get_fsm_id()、get_fsm_mode()、get_balance_mode()、get_swing_height()、get_stand_height()、get_phase()、get_odom()
h2	H2ArmActionClient	get_action_list()
h2	LocoClient	get_fsm_id()、get_fsm_mode()、get_balance_mode()、get_swing_height()、get_stand_height()、get_phase()、get_arm_sdk_status()、get_available_fsm_ids()
r1	AudioClient	get_volume()
r1	LocoClient	get_fsm_id()、get_fsm_mode()

“可构造”表示构造函数和 init() 已绑定，不表示没有机器人也能完成初始化或查询。

常见只读返回类型

方法	Python 返回类型
get_volume()	tuple[int, int]
get_state()	tuple[int, dict[str, str]]
get_image_sample()	tuple[int, list[int]]
get(name)	tuple[int, str]

方法	Python 返回类型
<code>check_mode()</code>	<code>tuple[int, str, str]</code>
<code>get_silent()</code>	<code>tuple[int, bool]</code>
<code>get_pkg_version()</code>	<code>tuple[int, str, dict[str, str]]</code>
<code>service_list()</code>	<code>tuple[int, list[ServiceState]]</code>
<code>is_tracking()</code>	<code>tuple[int, bool]</code>
<code>get_action_list()</code>	<code>tuple[int, str]</code>
<code>get_phase()</code>	<code>tuple[int, list[float]]</code>
<code>get_odom()</code>	<code>tuple[int, float, float, float]</code>
<code>get_available_fsm_ids()</code>	<code>tuple[int, list[int], list[str]]</code>

所有专属方法的完整参数、重载、C++ 来源和状态，请以 `.pyi` 与 `manifest` 为准。

Robot 预览类索引

以下索引帮助你在编辑器中定位类。出现于列表中不表示 `AVAILABLE`。

robot 根命名空间

```

ApplyLeaseData
ApplyLeaseParameter
ChannelFactory
ChannelNamer
Client
ClientBase
ClientChannelNamer
ClientStub
LeaseCache
LeaseClient
LeaseContext
LeaseServer
RequestFuture
RequestFutureQueue
Server
ServerBase
ServerChannelNamer
ServerStub

```

robot.a2

```

AudioClient
LedControlParameter
PathPoint
PlayStopParameter
PlayStreamParameter
PoseVec4
SportClient
TtsMakerParameter

```

robot.as2

```

PoseVec4
SportClient

```

robot.b2

```

BackVideoClient
ConfigClient
ConfigDelParameter
ConfigGetData
ConfigGetParameter
ConfigMeta
ConfigMetaData
ConfigMetaParameter

```

```

ConfigSetParameter
FrontVideoClient
JsonizeConfigMeta
JsonizeModeName
JsonizeSilent
LowPowerStatusData
LowPowerSwitchParameter
MotionSwitcherClient
PkgVersionData
RobotStateClient
ServiceState
ServiceStateData
ServiceSwitchData
ServiceSwitchParameter
SetReportFreqParameter
SportClient
stPathPoint

```

robot.g1

```

InternalFsmMode (enum)
AgvClient
AudioClient
G1ArmActionClient
JsonizeDataVecFloat
JsonizeVelocityCommand
LedControlParameter
LocoClient
MoveParameter
PlayStopParameter
PlayStreamParameter
TtsMakerParameter

```

robot.go2

```

ConfigClient
ConfigDelParameter
ConfigGetData
ConfigGetParameter
ConfigMeta
ConfigMetaData
ConfigMetaParameter
ConfigSetParameter
JsonizeCommObjInt
JsonizeConfigMeta
JsonizeDataBool
JsonizeDataDouble
JsonizeDataFloat
JsonizeDataInt
JsonizeDataString
JsonizeFlagBool
JsonizePathPoint
JsonizeQuat
JsonizeVec3
ObstaclesAvoidClient
ObstaclesAvoidMoveParameter
ObstaclesAvoidRemoteCommandSource
ObstaclesAvoidSwitchGetData
ObstaclesAvoidSwitchSetParameter
RobotStateClient
ServiceState
ServiceStateData
ServiceSwitchData
ServiceSwitchParameter
SetReportFreqParameter
SportClient
UtrackClient
UtrackSwitchGetData
UtrackSwitchSetParameter
VideoClient
VuiClient

```

stPathPoint

robot.h1

JsonizeDataVecFloat
JsonizeTargetPos
JsonizeVelocityCommand
LocoClient

robot.h2

FsmIdInfo
H2ArmActionClient
JsonizeArmActionCommand
JsonizeArmActionName
JsonizeDataVecFloat
JsonizeFsmIdList
JsonizeVelocityCommand
LocoClient

robot.r1

AudioClient
JsonizeDataVecFloat
JsonizeVelocityCommand
LedControlParameter
LocoClient
PlayStopParameter
PlayStreamParameter
TtsMakerParameter

版本、覆盖率与验证状态

文档对应版本

项目	值
文档日期	2026-08-28
Stub 分发包	unitree-sdk2-cpp-stubs
Stub 版本	0.3.0
Python 要求	>=3.10
Wheel 标签	py3-none-any
Manifest schema	1
Wheel SHA-256	126c3e43656de48a69f946a8d9faea87d396c983f91e4bfee48762144fac76ec

签名覆盖统计

指标	数量
IDL 类	64
IDL 属性	341
Robot 类	121
Robot 公共方法/构造签名	634
Manifest 条目	1213
AVAILABLE 条目	1168
SIGNATURE_ONLY 条目	45

状态数字应该怎样理解？

1168 AVAILABLE 不等于 1168 个机器人命令。它还包括：

- 533 个 IDL 值类型条目;
- DDS 生命周期 API;
- 构造函数;
- Robot Client 初始化和公共基础能力;
- 336 个已绑定 Client 方法;
- 80 个 Robot 参数/返回值类型、2 个 Lease 内存工具类和 1 个枚举。
- G1 的 5 个 CRC 重载条目和 7 个安全检查函数。

45 SIGNATURE_ONLY 集中在底层 Channel 命名器、抽象 Client/Server 基类、裸 ClientStub/Server Stub、RequestFuture 和服务端生命周期接口。这些接口依赖内部 DDS Request/Response 类型、工作线程或复杂回调所有权，不能只为提高覆盖数字而直接暴露。运动 Client 方法已经是 AVAILABLE，但仍由 MOTION_COMMAND 标签限制执行权限。

Robot JSON 值类型怎样使用？

72 个继承 SDK Jsonize 的参数和返回值类已经进入运行时。Python 字段直接读写同一个 C++ 对象；from_json() 和 to_json() 只负责跨越 Python 字典边界，字段解析和生成仍调用 Unitree C++ SDK 自己的 fromJson/toJson。

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataFloat

value = JsonizeDataFloat()
value.from_json({"data": 1.25})

print(value.data)      # 1.25
print(value.to_json()) # {"data": 1.25}
```

注意两个方向的签名不同：

方法	参数	返回值
from_json(value)	一个可 JSON 序列化的映射	None
to_json()	无	dict[str, Any]

LeaseContext 和 LeaseCache 也是 AVAILABLE，但它们只是进程内状态对象。构造和读写这两个类不会初始化 DDS；LeaseClient 和 LeaseServer 则是另一类会建立 SDK 生命周期状态的对象，不要混淆。

已完成的静态验证

当前签名包已通过：

- wheel 构建；
- 隔离环境安装；
- PEP 561 stub 发现；
- Mypy strict 类型检查；
- 安装后的类型推断检查；
- manifest 与生成器覆盖检查。

当前源码级测试结果：

```
33 passed, 4 skipped
```

跳过项需要导入 Linux 二进制扩展，macOS 源码检查环境无法执行。

Linux 二进制验证边界

在较早的 IDL 和 typed channel 修订上，Ubuntu 20.04 aarch64、Conda Python 3.10 环境已经验证过：

- 扩展成功构建和导入；

- 64 个消息类加载;
- 64 个 publisher/subscriber 类型注册加载;
- CycloneDDS 从 wheel 自带 .libs 解析;
- RUNPATH=\$ORIGIN/.libs;
- 测试结果为 22 passed。

加入当前完整 Robot Client 包装器后, 仍需要在目标 Linux 主机做一次新的:

1. 全量编译;
2. 导入测试;
3. 共享库依赖检查;
4. 336 个 Client 方法的注册检查;
5. G1 消息别名、CRC 和安全检查的运行时测试;
6. 不构造 Client、不初始化 DDS、不接触运动 API 的默认安全测试。

因此当前最准确的描述是:

```
IDL + typed channel: 已有目标机历史验证
Robot Client wrapper: 336/340 个公开方法已有源码
和签名, 等待当前修订目标机复验
G1 专项: 13 个消息别名、49 个 Client 方法、5 个 CRC
重载条目和 7 个安全检查等待目标机复验
运动/其他硬件副作用: 运行时入口已生成; 尚未做目标
机器人功能或安全验证
```

发布或交付前检查清单

- ☐ wheel 文件名和文档版本一致;
- ☐ SHA-256 与交付记录一致;
- ☐ api_manifest.json 的 summary 与测试一致;
- ☐ Mypy strict 通过;
- ☐ wheel 能在干净环境中安装;
- ☐ Linux 目标架构构建成功;
- ☐ import unitree_sdk2_cpp 成功;
- ☐ registered_message_types() 返回 64 项;
- ☐ ldd 没有 not found;
- ☐ 220 个运动方法都保留 MOTION_COMMAND 标签;
- ☐ 默认测试只检查方法注册, 不构造 Client 或调用命令;
- ☐ 默认测试没有初始化 DDS、联系硬件或命令运动;
- ☐ 硬件测试具有明确的人工启用和现场安全条件。

推荐学习路线

阶段 1: 只理解 Python 签名

目标: 不运行扩展也能正确写代码。

1. 安装 stub wheel;
2. 在编辑器中查看 MotorCmd 补全;
3. 用 Mypy 检查正确和错误赋值;
4. 学会查看方法的 AVAILABLE/SIGNATURE_ONLY 标签;
5. 学会查询 api_manifest.json。

完成标准: 你不会再把“Mypy 通过”理解成“机器人上能运行”。

阶段 2：理解消息对象

目标：在 Linux 运行时中安全创建和检查消息，不初始化 DDS。

1. 构建真实扩展；
2. 导入 `OsHelper`；
3. 创建 IDL 消息；
4. 练习标量和列表字段；
5. 练习嵌套对象“读取、修改、写回”；
6. 验证定长数组长度。

完成标准：你能解释为什么 `command.motor_cmd[0].q = ...` 可能不会保留。

阶段 3：理解 DDS，只使用自定义主题

目标：理解 channel 生命周期，不接触机器人控制 topic。

1. 查网卡和 domain；
2. 在 `tutorial/hello` 上创建 publisher；
3. 创建匹配的 subscriber；
4. 检查类型不匹配时的静态错误；
5. 练习 `try/finally` 关闭资源；
6. 练习回调只入队模式。

完成标准：你能画出 Publisher、topic、Subscriber 和 message type 的关系。

阶段 4：只读设备数据

目标：在安全环境中观察状态，不发控制命令。

1. 确认目标型号和固件；
2. 订阅只读状态 topic；
3. 记录消息频率和最近数据时间；
4. 调用 `manifest` 中标为 `READ_ONLY` 的 Robot Client 方法；
5. 正确处理状态码和超时；
6. 建立日志、关闭和异常策略。

完成标准：网络断开、服务超时或 Ctrl+C 时，程序能清晰退出并释放资源。

阶段 5：建立工程边界

目标：让上层业务与硬件实现解耦。

1. 用 `Protocol` 定义业务需要的最小接口；
2. 用 `fake/mock` 测试状态机和参数校验；
3. 把真实 Unitree Client 放在单独适配器中；
4. 让 CI 默认只运行无硬件测试；
5. 为 DDS、hardware、motion 测试使用明确标记；
6. 把 `manifest` 状态检查加入发布流程。

阶段 6：任何运动能力之前

运动接口已经绑定，但真实运动验证不是本文范围。进入这一阶段前，至少应具备：

- 对目标机器人 SDK 和控制模式的完整理解；
- 经过评审的上层安全适配器和执行策略，而不只是直接调用 binding；
- 目标架构二进制测试；

- 独立物理急停；
 - 机器人固定/吊装和隔离区域；
 - 边界值、通信丢失和超时测试；
 - 明确的现场负责人和启用流程；
 - 不会被普通单元测试或 CI 自动触发的硬件测试入口。
-

最后检查：我现在到底能做什么？

只安装了 stub wheel

你可以：

- 写所有签名中的 Python 代码；
- 获得编辑器补全和参数提示；
- 用 Mypy/Pyright 做静态检查；
- 查看完整 manifest；
- 为未来接口编写 mock 和上层逻辑。

你不可以：

- 执行 `import unitree_sdk2_cpp`；
- 创建真实 C++ 消息对象；
- 初始化 DDS；
- 连接机器人；
- 调用任何真实 Client。

安装了匹配的 Linux 扩展

你可以：

- 导入 `unitree_sdk2_cpp`；
- 使用 AVAILABLE 的 IDL、channel 和 Robot Client API；
- 在正确环境下订阅和发布已注册类型；
- 在正确设备上执行标为 `READ_ONLY` 的查询。

你仍不可以假设：

- `SIGNATURE_ONLY` 方法已经存在；
- DDS 初始化成功就一定能发现机器人；
- `write()` 返回 `True` 就代表设备完成了动作；
- 任何 AVAILABLE 运动方法已经针对你的型号、固件和现场完成安全验证。

一句话决策规则

先看签名，再看 manifest；
确认 AVAILABLE，再做 Linux 导入；
确认网络和服务，最后才谈硬件；
任何运动能力必须另行授权、评审和验证。